One-Dimensional Advection–Diffusion of a Dye Release

Explicit Upwind/Central Finite-Difference Solution in C, with Modified-Equation Analysis

Course Project 3 – OE5450 Numerical Techniques in Ocean Hydrodynamics (Tutorial 5, Jan–May 2025)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras
Faculty: Prof. K. Murali

Revised report, October 2026

Abstract

The transport of an instantaneous dye release ($C_0 = 10$ kg m^{-3} at one node) in a uniform stream is computed for $u = 0$, 1, 1.5 and 2 m s^{-1} with $D = 0.5$ m^{2}s^{-1}, using forward-Euler time integration, first-order upwind advection and central diffusion. The C code printed in the original report did not compile (a missing semicolon and an illegal initialiser on variable-length arrays); after these two corrections it reproduces the original results file exactly. Comparison with the exact Gaussian (Green's function) solution shows that the computed plume is systematically wider and lower than the physical one when $u > 0$. A modified-equation analysis attributes this to an artificial diffusion $D_{num} = \frac{1}{2}u\Delta x(1 - \mathrm{CFL})$, which equals 19% and 36% of the physical diffusivity at $u = 1$ and 2 m s^{-1}. Moment analysis of the computed fields confirms the prediction: the effective diffusivities obtained from $d\sigma^2/dt$ are 0.593, 0.639 and 0.676 m^{2}s^{-1} against the predicted 0.595, 0.639 and 0.680 m^{2}s^{-1}, and the plume centroid travels at u to within 0.1%.

Contents

1 Problem definition **3**
1.1 Exact solution . . . 3
1.2 Non-dimensional parameters . . . 3

2 Numerical method **3**
2.1 Discretisation . . . 3
2.2 Stability . . . 3
2.3 Modified equation and numerical diffusion . . . 4
2.4 Algorithm . . . 4

3 Code review and corrections **4**

4 Results **5**
4.1 Concentration profiles . . . 5
4.2 Moment analysis . . . 6
4.3 Original versus corrected code . . . 7

5 Discussion: how to reduce the numerical diffusion **7**

6 Conclusions **7**

A Source code (corrected) **7**

1 Problem definition

A dye is injected into a uniform one-dimensional flow. Its concentration $C(x,t)$ obeys

$$\frac{\partial C}{\partial t} + u\frac{\partial C}{\partial x} = D\frac{\partial^2 C}{\partial x^2}, \qquad -10 \le x \le 10 \text{ m}, \quad t > 0, \tag{1}$$

with $D = 0.5\ \mathrm{m^2 s^{-1}}$ and $u \in \{0,\ 1.0,\ 1.5,\ 2.0\}\ \mathrm{m\,s^{-1}}$ ($u = 1.5$ was the last case run in the original work). The dye is released once, at $t = 0$, as a single-node spike $C(0,0) = C_0 = 10\ \mathrm{kg\,m^{-3}}$, i.e. a mass per unit area $M = C_0\Delta x = 2\ \mathrm{kg\,m^{-2}}$. The ends are held at $C = 0$. (The code's title mentions a "continuous source", but the update never re-injects dye; the problem actually solved is an instantaneous release, which is also what the original plots show.)

1.1 Exact solution

For an infinite domain the solution is the advected heat kernel

$$C(x,t) = \frac{M}{\sqrt{4\pi D t}}\exp\left[-\frac{(x-ut)^2}{4Dt}\right], \tag{2}$$

whose moments are $\int C\,dx = M$, $\bar{x} = ut$ and $\sigma^2 = 2Dt$. It is valid for the finite domain as long as the plume ($\bar{x} \pm 3\sigma$) has not reached $x = \pm 10$ m.

1.2 Non-dimensional parameters

u (m/s)	0	1.0	1.5	2.0
CFL $= u\Delta t/\Delta x$	0	0.05	0.075	0.10
$d = D\Delta t/\Delta x^2$	0.125	0.125	0.125	0.125
cell Péclet $\mathrm{Pe}_\Delta = u\Delta x/D$	0	0.4	0.6	0.8
domain Péclet uL/D	0	40	60	80
$D_{num} = \frac{1}{2}u\Delta x(1-\mathrm{CFL})$ (m^2/s)	0	0.095	0.139	0.180
D_{num}/D	0	19%	28%	36%

Table 1: Grid ($\Delta x = 0.2$ m, 101 nodes; $\Delta t = 0.01$ s) and flow parameters.

2 Numerical method

2.1 Discretisation

$$C_i^{n+1} = C_i^n - \mathrm{CFL}\,(C_i^n - C_{i-1}^n) + d\,(C_{i+1}^n - 2C_i^n + C_{i-1}^n), \qquad i = 1,\dots,N_x - 2, \tag{3}$$

i.e. forward-time, backward (upwind for $u \ge 0$) space for advection and central space for diffusion, with $C_0^{n+1} = C_{N_x-1}^{n+1} = 0$.

2.2 Stability

With $\theta = k\Delta x$ the von Neumann amplification factor of (3) is

$$G(\theta) = 1 - (\mathrm{CFL} + 2d)(1 - \cos\theta) - \mathrm{i}\,\mathrm{CFL}\sin\theta, \tag{4}$$

and $|G| \le 1$ for all θ requires CFL$+2d \le 1$ (together with CFL$^2 \le$ CFL$+2d$). Here CFL$+2d \le 0.35$, so every case is stable; and since all coefficients of (3) are non-negative the scheme is also monotone (no new extrema, $C \ge 0$).

2.3 Modified equation and numerical diffusion

Taylor-expanding (3) about (x_i, t^n) and eliminating C_{tt} with the PDE ($C_{tt} = u^2 C_{xx} + \dots$) gives the equation that the scheme actually solves:

$$C_t + uC_x = \Big[D + \underbrace{\tfrac{1}{2}u\Delta x\,(1 - \mathrm{CFL})}_{D_{num}}\Big] C_{xx} + \mathcal{O}(\Delta x^2, \Delta t^2, \Delta x \Delta t). \tag{5}$$

The leading error is therefore an *extra diffusion*, from the first-order upwind difference ($+\frac{1}{2}u\Delta x$) partly cancelled by the forward-Euler time error ($-\frac{1}{2}u^2\Delta t$). The computed plume should behave like (2) with D replaced by $D_{eff} = D + D_{num}$. This prediction is tested in Section 4.2.

2.4 Algorithm

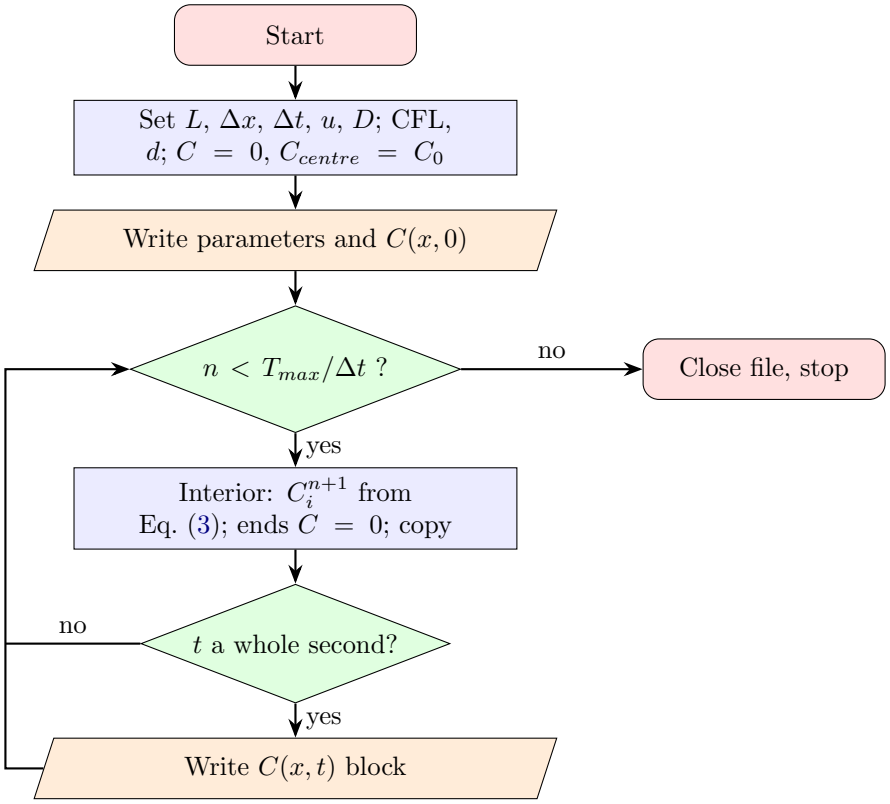


Figure 1: Flowchart of `code/advection_diffusion.c`.

3 Code review and corrections

The source code was taken from the original report (no separate source file was kept). Two compile errors were found and corrected; the method, parameters and output format are unchanged.

Location	Problem	Correction
`current_time = (n + 1) * DT`	Missing semicolon: compile error.	Semicolon added.
`double C_old[NX] = {0.0};`	`NX` is `(int)(L/DX)+1`, which is not an integer constant expression, so the arrays are variable-length arrays, which C does not allow to be initialised (compile error).	Declared without initialiser and zeroed in a loop.

Table 2: Corrections.

Points checked and left unchanged:

- The save test `fmod(current_time,1.0) <= DT/2` was verified to catch all 100 whole-second instants, because the time is formed as $(n+1)\Delta t$ rather than accumulated. An integer step test would be more robust if Δt were changed.
- In the original gnuplot script, block `index 0` is the initial condition but is labelled "Time = 1 s", so every curve in the original figures carries a label one second too late. The new plots are labelled from the time stored in each block.

With the two corrections the program reproduces the original `results.txt` ($u = 1.5\ \mathrm{m\,s^{-1}}$) exactly (maximum difference 0, Fig. 5), so the run that produced the original figures used a correct copy of the code and the errors were introduced only in the printed listing.

4 Results

4.1 Concentration profiles

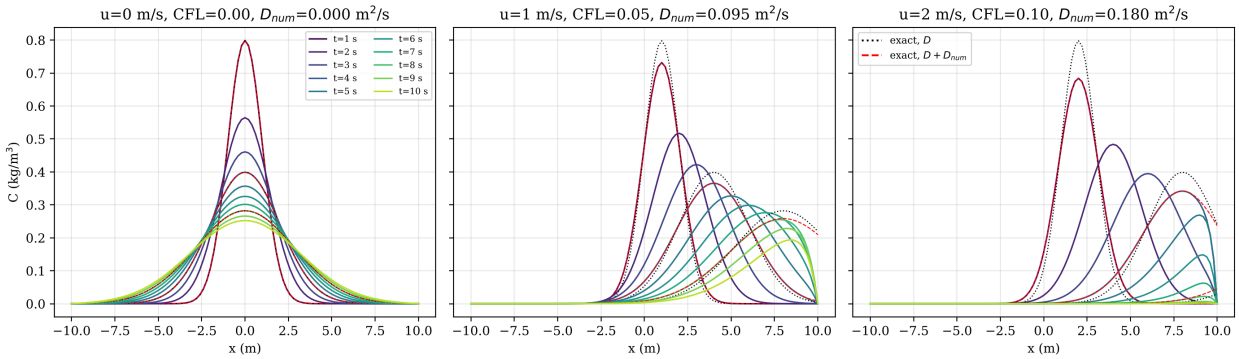


Figure 2: Computed profiles at $t = 1, \ldots, 10$ s (solid) and the exact solution with the physical diffusivity D (black dotted) and with $D_{eff} = D + D_{num}$ (red dashed) at $t = 1, 4, 8$ s.

For $u = 0$ the computed Gaussian coincides with the exact solution (L_2 error 5×10^{-5} at $t = 4$ s; peak 0.3991 vs. 0.3989). For $u > 0$ the computed plume is lower and wider than the physical solution but matches the exact solution evaluated with D_{eff}: at $t = 4$ s the peak is 0.3657 for $u = 1$ against 0.3989 with D and 0.3657 with D_{eff}. At $u = 2$ the plume reaches the outflow boundary after ≈ 4 s; the Dirichlet condition $C = 0$ then acts as a perfect sink and produces the sharp drop at $x = 10$ m. A zero-gradient outflow condition, or a longer domain, would be needed to follow the plume further.

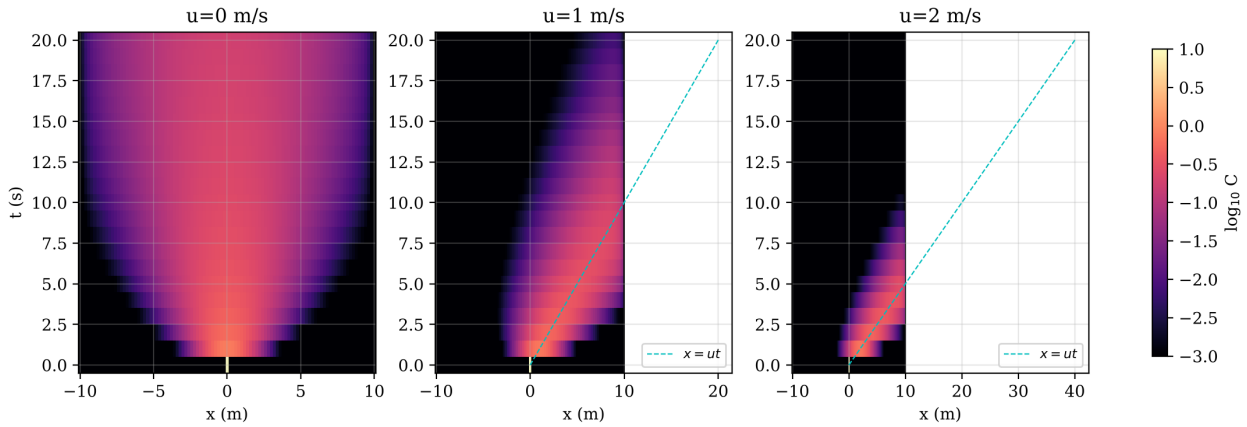


Figure 3: Space–time maps of $\log_{10} C$; the dashed line is the advective path $x = ut$.

4.2 Moment analysis

Mass $m_0 = \int C\,dx$, centroid $\bar{x}$ and variance σ^2 were computed from every stored profile. Linear fits over the times when the plume is fully inside the domain give:

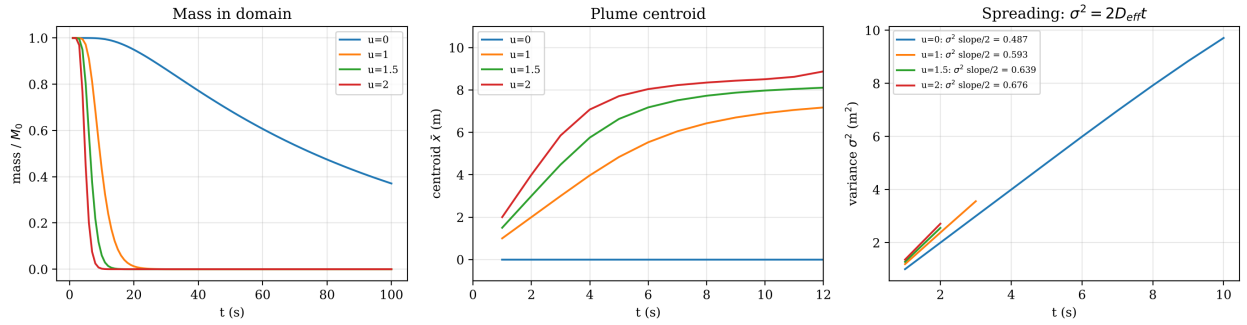


Figure 4: Mass remaining in the domain, centroid and variance.

u	D_{num} (theory)	D_{eff} (theory)	D_{eff} from $\frac{1}{2}d\sigma^2/dt$	Error	$d\bar{x}/dt$	Fit points
0	0	0.500	0.487	−2.5%	0.000	10
1.0	0.095	0.595	0.593	−0.4%	0.999	3
1.5	0.139	0.639	0.639	0.0%	1.500	2
2.0	0.180	0.680	0.676	−0.5%	1.999	2

Table 3: Effective diffusivity and transport speed ($\mathrm{m^2 s^{-1}}$, $\mathrm{m\,s^{-1}}$).

The measured spreading rates confirm the modified-equation prediction to within 0.5% for the advected cases; the small deficit at $u = 0$ comes from the slight loss of mass through the boundaries at $t \geq 9$ s ($m_0 = 0.997M$ at 10 s). The centroid speed equals u, i.e. the scheme has no phase error in the mean transport, and mass is conserved exactly until the plume reaches a boundary.

4.3 Original versus corrected code

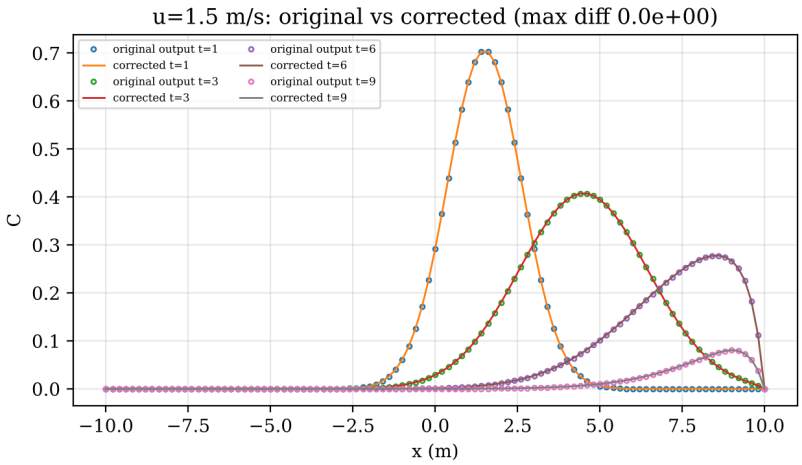


Figure 5: Original `results.txt` ($u = 1.5$) and the corrected code: identical.

5 Discussion: how to reduce the numerical diffusion

Since $D_{num} \propto u\Delta x$, the error can be reduced by (i) refining the grid, which costs $\Delta t \propto \Delta x^2$ through the diffusion limit; (ii) using a second-order advection scheme (central or Lax–Wendroff, stable here because $\mathrm{Pe}_\Delta < 2$), or (iii) a flux-limited TVD scheme that keeps monotonicity. With central advection the leading error becomes dispersive ($\mathcal{O}(\Delta x^2)C_{xxx}$) rather than diffusive.

6 Conclusions

- After two syntax corrections, the code reproduces the original results exactly and solves the advection–diffusion problem stably (CFL $+ 2d \leq 0.35$) and monotonically.
- The plume is advected at the correct speed and conserves mass, but first-order upwinding adds an artificial diffusivity $D_{num} = \frac{1}{2}u\Delta x(1 - \mathrm{CFL})$, i.e. 19–36% of D for $u = 1$–2 m s^{-1}. This explains why the original plots show lower peaks at higher velocity than physical diffusion alone would give.
- The modified-equation prediction is confirmed quantitatively by the second moment of the computed plumes (within 0.5%).

Reproducing the results

`bash run.sh` compiles `code/advection_diffusion.c` for each velocity (only the `U` constant is substituted), runs it and calls `code/postprocess.py`.

A Source code (corrected)

```c
#include <stdio.h>
#include <math.h>
//**********************ADVECTION-DIFFUSION SIMULATION WITH CONTINUOUS SOURCE**********************//
// Original author: Karthick J (OE23S018), OE5450 Tutorial 5. Code taken from the submitted report.
// Corrections are marked [FIX]. Note: the dye is released ONCE at t = 0 (instantaneous point
// release); the update does not re-inject it, so the 'continuous source' wording refers to the
```

```c
// initial condition only.
// Domain parameters
#define L 20.0 // Domain length (-10m to +10m)
#define DX 0.2 // Spatial step size (m)
#define Cd 10.0 // Initial concentration (kg/m^3)
#define D 0.5 // Diffusion coefficient (m^2/s)
#define DT 0.01 // Time step (s)
#define U 2.0 // Flow velocity (m/s)
#define T_MAX 100.0 // Total simulation time (s)
#define NX (int)(L/DX) + 1 // Number of spatial points
// Prototypes
void initialize(double C[], int center);
void update(double C[], double C_new[], double dx, double dt, double u, double
d, int center);
void save_results(FILE *fp, double C[], double time);
int main() {
  // Simulation parameters
  const int center = (NX-1)/2;
  const int num_steps = (int)(T_MAX/DT);
  double current_time = 0.0;

  // Concentration arrays
  // [FIX: NX is not an integer constant expression, so these are variable-length arrays,
  // which C does not allow to be initialised with '= {0.0}' (compile error). Zero them in a loop.]
  double C_old[NX];
  double C_new[NX];
  for (int i = 0; i < NX; i++) { C_old[i] = 0.0; C_new[i] = 0.0; }

  // Stability checks
  const double CFL = U * DT / DX;
  const double diffusion_number = D * DT / (DX*DX);

  // File handling
  FILE *fp = fopen("results.txt", "w");
  if(!fp) {
      printf("Error opening output file!\n");
      return 1;
  }

  // Write simulation parameters
  fprintf(fp, "# Advection-Diffusion Simulation Parameters\n");
    fprintf(fp, "# Domain Length: %.1f m\n", L);
    fprintf(fp, "# Velocity: %.1f m/s\n", U);
    fprintf(fp, "# Diffusion Coefficient: %.1f m^2/s\n", D);
    fprintf(fp, "# CFL Number: %.3f\n", CFL);
    fprintf(fp, "# Diffusion Number: %.3f\n\n", diffusion_number);
    // Initialization and save initial condition
    initialize(C_old, center);
    save_results(fp, C_old, 0.0); // Save initial condition at t=0
    // Main simulation loop
    for(int n = 0; n < num_steps; n++) {
        current_time = (n + 1) * DT; // [FIX: missing semicolon]
        // Update concentrations
        update(C_old, C_new, DX, DT, U, D, center);

        // Save data at specified intervals
        if(fmod(current_time, 1.0) <= DT/2.0) {
            save_results(fp, C_old, current_time);
        }
    }
    fclose(fp);
    printf("Simulation completed successfully!\n");
    return 0;
}
void initialize(double C[], int center) {
    for(int i = 0; i < NX; i++) {
        C[i] = 0.0;
            // Continuous source at center
    }
    C[center] = Cd;
}
void update(double C[], double C_new[], double dx, double dt, double u, double
d, int center) {
    // Update interior points
    for(int i = 1; i < NX-1; i++) {
```

```
        const double adv = u * (C[i] - C[i-1]) / dx;
        const double diff = d * (C[i+1] - 2*C[i] + C[i-1]) / (dx*dx);
        C_new[i] = C[i] - dt * adv + dt * diff;
    }
    // Boundary conditions
    C_new[0] = 0.0; // Upstream Dirichlet BC
    C_new[NX-1] = 0.0; // Downstream Dirichlet BC
    // Update array
    for(int i = 0; i < NX; i++) {
        C[i] = C_new[i];
    }}
void save_results(FILE *fp, double C[], double time) {
    fprintf(fp, "# Time = %.2f s\n", time);
    fprintf(fp, "%-12s %-12s\n", "Position(m)", "Concentration");
    for(int i = 0; i < NX; i++) {
        double x = -L/2 + i * DX;
        fprintf(fp, "%10.2f %14.4f\n", x, C[i]);
    }
    fprintf(fp, "\n\n"); // Two blank lines for gnuplot index
}
```